Chapter #11

Indexes for the Data WAREHOSE.

⇒ Indexes are way to organize data.

⇒ They allow to access specific data quickly without going through all the data.

⇒ They enhance the performance.

⇒ Index key stand for the set of attributes upon which the index was built.

⇒ it help to access relations tuples quickly.

B+ Tree indexes

⇒ This is a data structure, fancy way of accessing data organizing

- B-Trees are like trees with branches and leaves.
- Each branch points to a specific piece of data, and the leaves contain the actual data.

How B-tree works?

⇒ when finding something in a B-Tree, it starts at the top (root) and follows the branches until it reaches the right leaf. This is faster than searching through every piece of data one-by-one.

B-Tree Properties:

Orders:

- The order of B-Tree determines how many children

Leaves:

- All the data is stored in the leaves of the B-tree.

Pointers:

- Pointers connect the nodes of the B-tree, allowing for efficient navigation

Searching in B-Trees

Sequential search:

- Start at one leaf and following the pointers to the next leaves.

Interval search:

- Specific value can search by following the branches of B-trees.

Notes:

⇒ B*-trees evolved from B-trees (Bayer and McCreight, 1972).

Types:

⇒ B*-trees can be either:

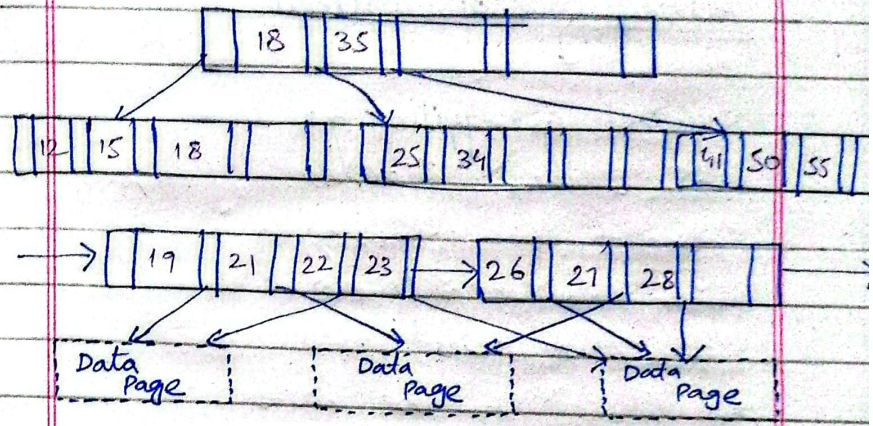
- Primary
- Secondary

Primary index:

- This is built on the primary key of a table, which is a unique identifier for each row. Each leaf in a primary index points to exactly one row.

Secondary index:

- This is built on any other attribute of a table. Each leaf in a secondary index can point to multiple rows that have the same value for that attribute.



Bitmap indexes

- A bitmap index like a matrix with row and columns.
- Each row represents a tuple, and each column represents a distinct value of an attribute

How bitmap indexes work?

- A 1 in the matrix indicates that the tuple in that row has the attribute value associated with that column.

Advantages of Bitmap indexes:

- Efficient for filtering:
  ⇒ Bitmap indexes are very efficient for filtering data based on specific attribute values.
  (Use Boolean operators)
- Space: efficient
  ⇒ They can be more space-efficient than other types of indexes, especially for columns with low cardinality.

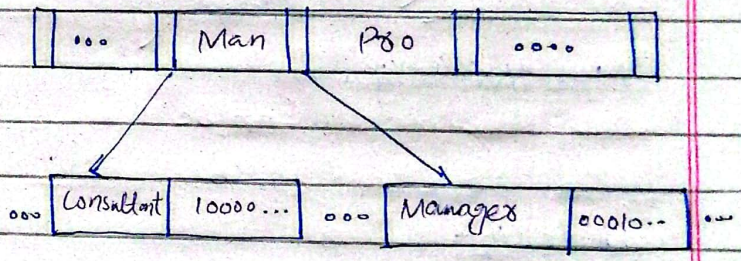
Optimization:

Organizing Bitmaps as sets of vectors:

By organizing bitmaps as sets of vectors, you can reduce the amount of data that needs to be loaded from disk when searching for specific values.

Structure:

Structure of Bitmap indexes are identical to B*-trees. Except that bitmap index tree leaves contain bit vectors in the place of RID lists.



B-Tree vs Bitmap

Structure:	
• Tree like structure with nodes & pointers	• Matrix of bits representing attribute values.
Data storage:	
• Stores data in nodes.	• Stores bitmaps

	representing data presence/absence
Query Types:	
○ Efficient for exact match and range queries	○ Efficient for filtered based on multiple conditions.
Scalability:	
○ Scales well with increasing data volume	○ Scales well for columns with low cardinality.
Space efficiency:	
○ Generally more space-efficient	○ Can be or can not be.
Complexity:	
○ Complex to implement	○ Simpler to implement
Best use cases:	
○ General purposes, especially for transactional databases	DWH & analytical applications, especially for filtering & aggregation.

Advanced Bitmap indexes:

Bit-sliced indexes:

- a type of advanced bit-map index used to organize data in database.
- A bit-sliced index is a matrix with rows and columns.
- Each column of index (slice) is stored separately.

Pros:

- Efficient for numeric attributes, because they efficiently calculate aggregate values.
- Scalability
- Complex queries

Note:

- Standard bitmap indexes linearly get larger as the number of distinct key values increases.
- Bit-sliced indexes show a logarithmic growth.

Bitmap-encoded indexes

- Same like matrix with rows and columns.
- But bitmap-encoded indexes can be used for both numeric and non-numeric attributes, making them versatile.

Projection Indexes

Structure:

- A projection index as a list of values for a specific attribute.
- This list is ordered in the same way as the rows in the table.

Advantages:

- Efficient for retrieving attribute values.
- Reduced disk Access.

Limitations:

- Efficient for fixed length ~~and~~ values.
- Require additional storage space.

Join indexes

Structure:

⇒ A list of pairs of tuple IDs.

⇒ Each pair represents two tuples from different tables that are related to each other

Advantages:

- Efficient for joins:-
- Reduced disk access

Limitations:

- Space overhead
- Maintenance overhead.

Star-Join indexes

a list of tuples, each containing a set of Row IDs

from different tables.
- Efficient scan without scanning the entire tables.

Advantages:
- Efficient for joining multiple tables.
- Reduced disk access.

Spatial indexes

→ A type of data structure used to organize and query spatial data in databases.

→ Spatial indexes are designed to optimize the storage and retrieval of spatial data.

→ They organize spatial data in a way that allows for efficient searching and querying.

Common-Techniques:
- R-Tree
- Z-order curve
- Quadtree

Joining Algorithms

Joining algorithms are techniques used to combine rows from two or more tables based on a related column.

Some common join algorithms:

Nested Loop Join:
- Best for small datasets.
- Simple but inefficient -
- iterates over each row of the outer table and compares it to every row of the inner table.

Hash join:
- well suited for equi-joins.
- Efficient for large datasets.
- Creates a hash table for one relation & probes the other relation against it.

Sort - Merge join:

- Sorts both relations on the join attribute.

⇒ Merges the sorted relations to produce the result.

⇒ Efficient for large sorted relations.

Hybrid Hash join:

⇒ Combine hash join & sort-merge join.

⇒ Divides the larger relations into partitions and processes them in memory or on disk.

Choosing the right algorithm:

⇒ The choice of joining algorithm depends on various factors:

- Table Sizes
- Data distribution
- Memory availability
- Index availability